

LEGASPI, KENT HALE A.

CS301 – CS ELECTIVE 1: CLOUD COMPUTING DEVELOPMENT

LIVE LINK: <https://the-hustle-hub.vercel.app/>

GITHUB REPOSITORY: https://github.com/khey25/legaspi_prelimfastapi

Part 1: Technical Documentation (What You Did)

Write a paragraph for each section answering these specific guide questions, accompanied by proof-of-work screenshots:

1. API Design & Data Choice:

Guide Questions: What custom domain/topic did you choose, and why? What dataset dictionary keys did you define in your index.py, and why are those attributes important for describing your items?

- For our Cloud Computing Project, I particularly chose to document and create a website for money making methods in games. I remember hearing from you that I can't just focus on making a website about money making methods on one game which is "gta online". Which made me think that I don't have to scrap my project totally, I can explore and expand my library with more games that has information on how to make money on those games.
- Anyways, In my gta.py file, I set up a custom dataset using dictionary keys like businesses, heists, and contact. Inside those categories, I tracked specific attributes that actually matter to players, like setup_cost, max_payout, solo_friendly, and cooldown_minutes. I chose these because if you are grinding in a game, you need to know exactly how much money you are making, how much it costs to start, and if you can run it solo without wasting time.

```

    "Suspiciously Profitable": [
      {
        "name": "Acid Lab",
        "property": "Brickade 6x6 (Freakshop)",
        "associated_network": "Fooliganz",
        "setup_cost": 750000,
        "is_passive": True,
        "solo_friendly": True,
        "max_payout": 335200,
        "restock_method": "Buy or Steal",
        "can_be_raided": False,
        "cooldown_minutes": 0,
        "min_players": 1,
        "max_players": 4,
        "release_update": "Los Santos Drug Wars",
        "release_year": 2022,
        "location_options": 1,
        "image_url": "https://github.com/khey25/legaspi_prelimfastapi/blob/main/gta/gtaimages/acid.jpg?raw=true"
      }
    ]
  }
}

```

2. Backend Endpoints & Implementation:

Guide Questions: How did you modify your routes in index.py? How does your search function filter through your custom text fields?

[Include Screenshot 1: Your live /docs page showing your custom endpoints.]

[Include Screenshot 2: Direct browser view of your raw JSON endpoint output.]

- To keep my code organized, I split my backend into modules using FastAPI's `APIRouter()` in `gta.py`, and then plugged that into my main `index.py` file with a `/gta` prefix. For the search feature, I built endpoints like `/gta/businesses/search`. The way it works is pretty straightforward, it takes whatever the user types into the search bar, loops through the database, and uses Python to check if that text matches any part of the name attribute in my items. If it finds a match, it grabs that specific data and sends it back to the frontend.

index.py X

api > index.py > ...

```
1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3
4  # Import your GTA routes
5  from api.gta import router as gta_router
6
7  app = FastAPI()
8
9  # --- CORS MIDDLEWARE ---
10 app.add_middleware(
11     CORSMiddleware,
12     allow_origins=["*"],
13     allow_credentials=True,
14     allow_methods=["*"],
15     allow_headers=["*"],
16 )
17
18 # --- PLUG IN THE ROUTERS ---
19 # This automatically slaps "/gta" in front of every route in gta.py!
20 app.include_router(gta_router, prefix="/gta", tags=["GTA"])
21
22 # (When you build Warframe later, you will add it right here!)
23 # from api.warframe import router as warframe_router
24 # app.include_router(warframe_router, prefix="/warframe", tags=["Warframe"])
25
26 # --- ROOT ENDPOINT ---
27 @app.get("/")
28 def root():
29     return {"message": "Welcome to the Gaming Hustle API! The server is running perfectly."}
```

```
gta_database = {
    "businesses": {
        "Suspiciously Profitable": [
            {
                "name": "Acid Lab",
                "property": "Brickade 6x6 (Freakshop)",
                "associated_network": "Fooliganz",
                "setup_cost": 750000,
                "is_passive": True,
                "solo_friendly": True,
                "max_payout": 335200,
                "restock_method": "Buy or Steal",
                "can_be_raided": False,
                "cooldown_minutes": 0,
                "min_players": 1,
                "max_players": 4,
                "release_update": "Los Santos Drug Wars",
                "release_year": 2022,
                "location_options": 1,
                "image_url": "https://github.com/khey25/legaspi_prelimfastapi/blob/main/gta/gtaimages/acid.jpg?raw=true"
            }
        ]
    }
}
```

```
const searchBar = document.getElementById('search-bar');
searchBar.addEventListener('input', async (e) => {
  const searchTerm = e.target.value.trim();

  try {
    const response = await fetch(`${API_URL}/gta/businesses/search?q=${searchTerm}`);
    const data = await response.json();

    document.getElementById("grid-profitable").innerHTML = "";
    document.getElementById("grid-mid").innerHTML = "";
    document.getElementById("grid-bad").innerHTML = "";

    if (data["Suspiciously Profitable"]) buildCards(data["Suspiciously Profitable"], "grid-profitable");
    if (data["Above Minimum Wage"]) buildCards(data["Above Minimum Wage"], "grid-mid");
    if (data["Going Bankrupt"]) buildCards(data["Going Bankrupt"], "grid-bad");
  } catch (error) {
    console.error("Error searching API:", error);
  }
}
```

3. Vercel Deployment Process:

Guide Questions: What directory structure did you set up so Vercel could recognize your code as a serverless function? What happened when you pushed your customized code updates to GitHub?

- Getting this live on Vercel meant I had to set up my folders in a very specific way so it knew I was running a Python backend instead of just a basic website. I put my Python files (index.py and gta.py) inside an api/ folder, and left all my static frontend HTML/CSS, my requirements.txt, and a custom vercel.json routing file right in the main root folder. Once that was set up, deploying was super easy. Every time I pushed my code to GitHub, Vercel automatically caught the update and rebuilt the live website instantly without me having to manually restart any servers.

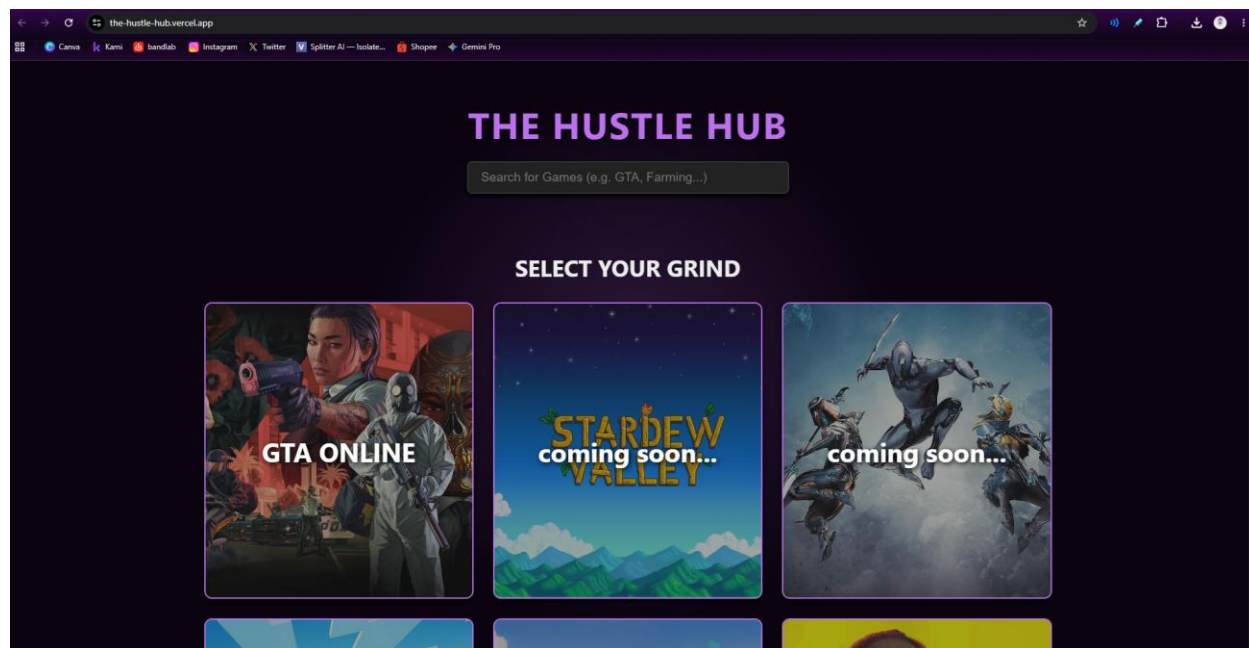
 khey25 Add files via upload ✓		968ee99 · 3 days ago	🔄 88 Commits
📁 .github/workflows	Create static.yml	2 weeks ago	
📁 api	Update gta.py	5 days ago	
📁 gta	Update business.js	3 days ago	
📁 menuimages	Add files via upload	3 days ago	
📄 index.html	Update index.html	3 days ago	
📄 menu.css	Add files via upload	3 days ago	
📄 menu.js	Add files via upload	3 days ago	
📄 requirements.txt	Add files via upload	2 weeks ago	
📄 vercel.json	Update vercel.json	3 days ago	

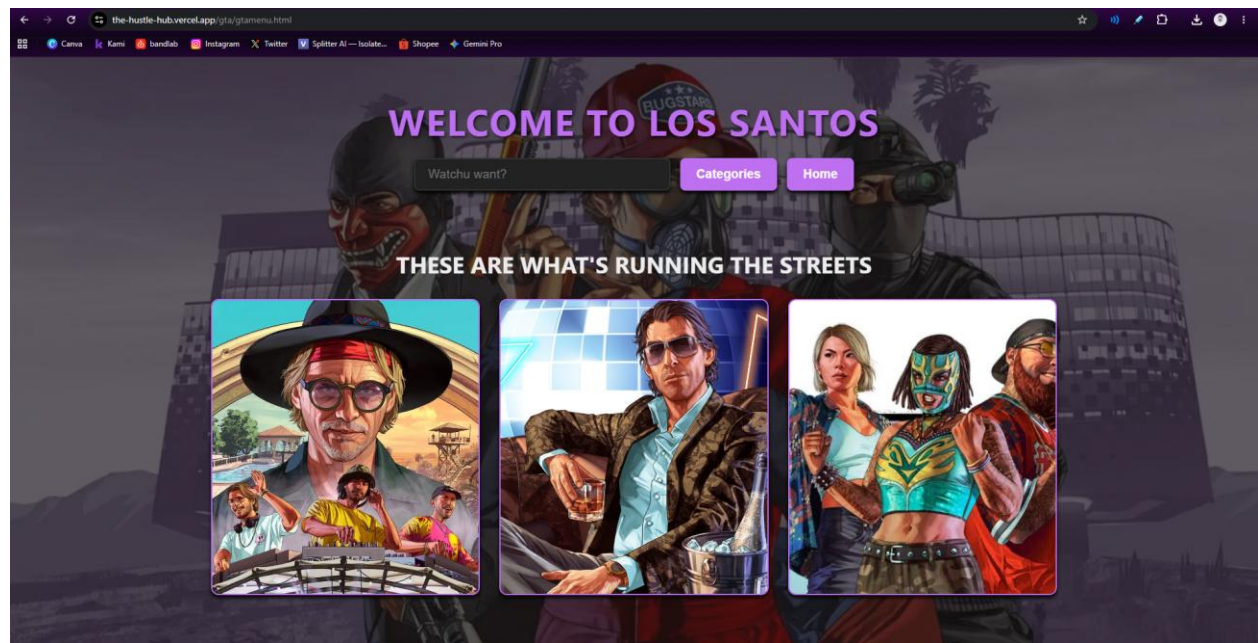
4. Frontend Integration:

Guide Questions: How did you update app.js to connect to your live Vercel URL instead of running locally? How did you modify your JavaScript template literals to render your new custom dataset onto the web page?

[Include Screenshot 3: Your finished customized web application running in a browser with your new theme loaded.]

- To get my frontend talking to the live database, I went into my JavaScript files and changed the API URL from my local testing server to my new live Vercel domain (<https://the-hustle-hub.vercel.app/>). Once the JSON data was successfully flowing in, I used JavaScript template literals to actually build the content on the screen. By grabbing attributes from the database using syntax like `${item.name}` and `${item.max_payout}`, my script dynamically generates the HTML data cards and drops them right onto the page to match the neon hub theme.





Part 2: Technical & Personal Reflection Questions

Answer the following questions thoughtfully at the end of your document:

Technical Core Questions:

1. Why do we need to specify CORS (Cross-Origin Resource Sharing) middleware in FastAPI when connecting our local frontend?
 - because web browsers are incredibly paranoid about security. if your frontend tries to request data from a backend API running on a completely different port or server, the browser will instantly block it to prevent malicious cross-site attacks. Adding the CORSblock in our index.py file is basically handing our frontend a VIP pass. By setting `allow_origins=["*"]`, we are explicitly telling the browser to trust our api
2. Trace the path your web application takes when a user enters text into the search bar and clicks "Search" (from the JS `fetch()` request to Vercel and back to the DOM).
 - When a user types a keyword into the search bar and hits enter, it travels through internet. First, the frontend JavaScript's `fetch()` request, attaching that keyword to our live Vercel URL. When that request hits the live server, Vercel checks the rewrite rules we set up in our `vercel.json` file and forwards the request straight to our Python backend instead of looking for a static HTML file. From there, FastAPI catches it and routes it to our specific search

endpoint inside `gta.py`. Python quickly loops through our database dictionary, filters out the exact matches based on the search word, and packages the data into a neat JSON format. Vercel then gives that JSON payload right back to the user's browser. Finally, our frontend JavaScript intercepts that incoming data, wraps the details in HTML template literals, and put it directly into the DOM.

Experience & Learning Reflection Questions:

3. What was the most challenging bug or issue you encountered while deploying to Vercel or linking your backend to your frontend, and how did you resolve it?
 - The most annoying bug we ran into is 404: NOT_FOUND error right after deploying. My Python API was running fine, but Vercel literally couldn't find my `index.html` file, so the main website was just a blank error screen. I resolved this by updating the `vercel.json` file to include a second rule in the build list using `@vercel/static`. This explicitly told Vercel to preserve my HTML, CSS, and JS files alongside the Python backend.
4. How did seeing your API running live on a public cloud URL (Vercel) compare to running code locally on your own computer?
 - Seeing a live link deployed from a public cloud URL feels like a good progress on my end. From our college journey, We only typically launch websites or app from our local devices overtime, and seeing a live link that people can click at anytime without me having to worry is a different feeling and an achievement.
5. FastAPI automatically generates interactive documentation (`/docs`). How did having this auto-generated tool impact the way you built and tested your endpoints?
 - Having the `/docs` page generated automatically was a convenient timesaver. Instead of guessing if my Python routing was correct, I could just go straight to that `/docs` URL and test everything visually.
6. If you were given more time to expand this full-stack project, what additional endpoints, HTTP methods (e.g., `POST` or `DELETE`), or UI features would you add next?
 - If I had more time, the next immediate step would be building out the specific database endpoints for Stardew Valley and Warframe, since I already have their UI cards ready to go on the main hub.

7. In your own words, why is separating the backend API from the frontend website beneficial when designing modern web applications?

- Separating backend API from Frontends are definitely a necessary in modern web applications. Because modern web applications typically carry a lot of coding and data inside them. Designing a UI should be separated from a backend database where you rarely modify. If I want to change the neon background or change the CSS on the frontend, I don't have to worry about accidentally breaking the Python database. keeping the API entirely separate, I could build a mobile app version of The Hustle Hub in the future, and it could grab data from the exact same backend without me having to rewrite any of the database code.